Agent 架构

如何理解 agent harness

【面试官】可以介绍一下你目前对 Harness 的理解吗？它到底解决了什么样的问题，又是怎么演变而来的？

【候选人】我理解 Harness 不是一下子出来的，它是逐步演化出来的概念，所以理解 Harness 就要理解它的演变历史。整体上，我会把它理解成：从工程上保证 Agent 能够有效、持久、正确、稳定、高效完成指定任务的一整套工程框架。

第一阶段：纯对话式大模型。
最开始 GPT 火的时候，就是对话式的大语言模型，你问一句它答一句，和外界没有任何交互。

第二阶段：引入工具调用，Agent 诞生。
Agent 和纯大模型最大的区别是什么？我觉得就是 Agent 可以调用工具、可以和外界交互，这是迈出的最大一步，也是最核心的差别。但这个时候的做法还很简单，最原始的就是 Function Calling：把工具参数解析出来，让它去执行，就完了。

第三阶段：出现问题，开始做 Prompt 工程和上下文工程。
用起来之后问题就出现了：一个是指令偏差，模型本身有幻觉性，不一定能完全按照要求执行，一开始的解法就是写更好的 Prompt；另一个是执行效果不好——很多时候不是模型能力不行，而是它没获取到必要的、有效的信息。所以就开始做上下文工程：执行每个任务的时候，给它恰到好处的信息，且要精简，不然它会迷失、窗口过大会爆掉。但这个时候，其实还只是停留在信息管理层面的优化。

第四阶段：Harness 登场。
再往后，最大的问题就是：哪怕给了所有必要信息，调用工具还是会出错，而且出错是常态、是动态变化的——工具调用会出错，模型调 API 也会出错，总不能一出错就直接停下来。Harness 就是针对这个问题衍生出来的：怎么管理错误、怎么纠偏，让它就算出错之后也能恢复到正常运行状态，继续按照预定的任务目标执行下去。要注意，Harness 和前面这些不是互相取代的关系，是包含的关系，它是把前面的东西都包进来，再从工程上又推进了一层。

所以我的理解是：**Harness 就是从工程上保证 Agent 能够有效、持久、正确、稳定地完成指定任务的一整套工程框架。**

比如工具的执行流程，原来直接输出 Prompt、直接输出模型调用、工具调用就完了。现在我们加一层：解析 → 校验 → 权限管理 → 执行 → 最关键的结果结构化处理，再把结构化的结果给大模型，让它去分析、去自己修复问题，这是一个优化。

从最简单的 Function Calling 说起：把工具参数解析出来，让它去执行，这是最原始的方式。但它应该是一套完整的流程。比如流式输出的时候先解析出要执行的工具，这也是加快速度的一方面。

但最核心的是，工具执行正常、结果正常当然无所谓；如果出错了，要把工具错误的结果也以结构化的形式反馈回去——到底是参数传错了、信息不足，还是最简单的 API 调用有问题，都要把错误信息用结构化数据包装起来返回给大模型。

相当于用 ReAct 的方式：参数传错了，就看看是不是传其他参数；API 调用错误、限流了，就等一下重试；文件路径找不到，就泛化搜索一下，是不是只是一个字母的错误，列出相似的文件列表再选。就是在这样的循环里做处理。

当然还有剩下的，比如安全权限防护、上下文保护、上下文组装，甚至记忆管理，日志监控等等，这些都是 Harness 要做的事。还有怎么设计工具描述，让大模型准确理解工具内部的操作逻辑，对吧。

Harness 做的就是：出错了怎么纠偏，怎么让它继续按照预定的任务目标执行下去。我是这样理解的。

【面试官】：有一个问题，Agent 里面包含哪些部分呢？

Agent 包含哪些部分？我觉得可以顺着“从内核到外围”的顺序来讲。

第一层：核心引擎——大模型 + 工具调用

Agent 最核心的、和对话大模型最核心的差别，就是工具调用。有了工具，Agent 就可以和外界进行交互，这是最核心、最差异化的一点。可以这么理解：大模型本身是“大脑”，负责理解和决策；工具是它的“手脚”，负责真正去执行动作。

第二层：规范与上下文层——让大脑正确地决策

有了工具这个基础之后，光有手脚还不够，还得有规范约束和信息组织，这里衍生出几个组件：

- **System Prompt 系统提示**：定义角色和行为规范；
- **Skill 技能、按需加载**：不把所有能力一次性塞进上下文，用的时候再挂载；
- **上下文管理**：这是把所有信息组装起来的地方——相当于把系统提示、工具的 Skill、数据，还有对话历史，组装成最终的输入。

第三层：工程整合层——Harness

再往外一层，就是前面提到的 Harness：本质上就是把上面所有组件组合在一起，使 Agent 能够持久、正确地完成任务。

落到具体实现上，最典型的是一套工具的完整执行模块：

解析 → 校验 → 安全权限管理 → 工具执行 → 工具结果的结构化返回

第四层：扩展与保障层

在这个基础上还有一些延伸出来的组件：

- **子 Agent**：可以把指定的任务，用工具的方式去启动一个子 Agent 去执行，实现任务分解；
- **日志监测、结果反馈**：让整个运行过程可观测、可回溯。

这些组件合在一起，目的都是同一个：让 Agent 能够持久、正确、稳定地运行，完成我们设定的目标或者任务。

agent 流程/如何减少 agent 首 token 输出的响应时间

首 Token 耗时主要由三部分构成：**Prompt 预填充计算耗时 + 系统调度 / 上下文组装耗时 + ReAct 场景下的工具调用等待耗时**，我会从底层到应用层分层优化：

① 模型推理层：优化预填充速度，比如前缀 KV 缓存、PagedAttention 分页缓存；

② 应用架构层：精简上下文、做意图路由、按需加载组件；

③ ReAct 场景层：异步并行工具、前置轻量判断，减少串行等待。

```
用户请求
  ↓
轻量请求分类器
  ↓
是否需要复杂 Agent?
  ├── 否：直接调用低延迟模型，立即 streaming
  └── 是：
      并行执行：
        - 加载用户/会话/权限
        - RAG 检索
        - 工具候选选择
        - prompt prefix cache 查询
      ↓
      构造最小 prompt
      ↓
      调用模型，streaming 输出
      ↓
      需要工具时再进入 tool loop
```

如果要按投入产出排序，我会这样排：

1. **缩短首轮 prompt**：减少历史、工具 schema、RAG 内容。
2. **启用 streaming 并检查全模块 flush**。
3. **并行化 Agent 前置 I/O**：用户信息、历史、RAG、权限、配置。
4. **简单请求绕过 Agent**：不需要工具/RAG/规划就直接调用模型。
5. **动态模型路由**：简单任务用低延迟模型。
6. **prefix / prompt cache**：缓存系统提示词、工具定义、长期记忆。
7. **减少首 token 前工具调用**：能规则判断就别先问模型。
8. **推理服务低延迟调度**：短请求专用池、降低 batch wait。
9. **RAG 优化**：query cache、topK 降低、rerank 延后或裁剪。
10. **前端和网关避免缓冲**。

Agent 场景下降本增效的手段

把“降本”理解成三件事一起降：模型调用成本、工具/RAG 成本、失败重试成本。真正有效的策略，不是单纯换小模型，而是让便宜路径先跑，只有不确定时才升级到贵路径。

任务分层路由

先用规则或小模型判断任务难度。简单问答、固定流程、结构化抽取，直接走便宜模型或模板；复杂推理、长链工具调用，再进重 Agent。这个通常是最大头的降本。

压缩上下文

这是最直接的 token 降本手段。做法包括：历史窗口裁剪、长期记忆摘要、去重、只保留最近有效轮次、工具 schema 精简、RAG 片段只取最相关的少量内容。上下文越短，推理越便宜，首 token 也通常更快。

减少工具和检索的无效调用

不要“默认先检索、默认先调用工具”。先做轻量判定，只有真的需要时才查知识库或调外部系统。RAG 里可以先做 metadata 过滤，再向量检索，再 rerank，不要全量重排。

控制 Agent 轮次

很多成本浪费在“问模型，模型决定工具，工具返回，再问模型”的循环里。可以设定工具调用预算、最大迭代轮数、超时预算，避免无限思考和多轮试错。

缓存

适合缓存的东西很多：系统 prompt 前缀、工具定义、常见检索结果、用户资料、权限判断、会话摘要、embedding 结果、热门问答。缓存命中率一高，成本会很明显下降。

减少输出长度

输出 token 也是成本。对不同场景设合理上限，要求结构化输出，避免长篇空话。很多 Agent 场景其实只需要简洁结论、步骤、或 JSON。

推理服务层优化
如果是自建推理，这部分很重要：prefix cache、KV cache 复用、动态 batching、短长请求分池、量化、speculative decoding、warm pool、避免冷启动。
如果是外部 API，这部分你们控制较少，但前面的上下文和路由优化依然能直接省钱。

用微调或蒸馏减少 prompt 和工具开销
当一个 Agent 流程非常固定，且重复率高，微调往往能让模型少看很多说明文字，少走很多工具判断，长期看很划算。但它通常不是第一步，先把路由、上下文、缓存做完再考虑。

做 Agent 落地最核心的是什么？是流程编排，还是具体模块的设计？

【面试官】
做 Agent 落地最核心的是什么？是流程编排，还是具体模块的设计？

【候选人】
我觉得要分成两部分来看，结合我做的 A 项目来说，会更清楚。

第一点，也是首要的，是业务抽象与流程拆解。
现在 Agent 的通用架构其实已经很成熟了，甚至用 Claude 几分钟就能生成一个基础 Agent 框架，但核心是能不能真正在业务里用起来。

做 Agent 之前，先要把业务流程拆透：
哪些步骤是固定的、确定性的，就用脚本、硬编码流程实现，不用大模型；
哪些环节需要判断、推理，再交给大模型、用 Agent 循环去做。
不能幻想大模型搞定一切，把业务和技术的边界划清楚是第一步。

而且部分同事有时对 AI 不熟悉，会对大模型有不切实际的预期，我们需要帮他们把业务逻辑拆解、抽象成 Agent 能落地的流程。

其次才是垂类 Agent 的架构设计与优化

业务流程拆清楚之后，再搭高效的 Agent 循环。

通用框架，比如 Claude Code，问题有几个：
一是 System Prompt 非常长，包含了安全、越狱防护等大量通用约束，垂类场景根本不需要，既浪费 Token、提升成本，还会降低指令遵循效果；
二是工具全量加载，Claude Code 光工具定义就占 15% 左右的上下文窗口，垂类场景只需要特定工具，全量加载既冗余又影响效果。

此外还有 Skill 的元数据。
我们的优化思路是：
精简 System Prompt，只保留垂类相关的约束；
做自定义工具，只保留业务需要的特定工具，并且在工具内部提前做防冲突、防重复执行的逻辑，减轻 Agent 调度的负担。

另外，垂类场景里，记忆系统非常重要。
没必要为了垂类场景微调模型，把专家经验沉淀到记忆里，做领域知识的蒸馏，效果提升会很明显，成本也更低。

Agent 产品对比

你觉得 Claude-Code 和 openclaw 里面哪些设计比较巧妙？有没有什么缺点？

Claude Code 的优缺点

优点

完整的 tool call 全模块。 流式输出识别工具调用指令，不用等完整输出；工具输出格式前置校验；工具权限分级；无论成功失败，都结构化返回详细错误信息给模型，让模型判断下一步动作。

上下文经济与工具延迟加载。 Claude Code 使用模块化 system prompt 配合缓存感知边界，约 40 个工具通过插件架构管理。它通过 defer_loading 机制实现工具延迟加载，避免所有工具定义一次性占满上下文窗口。每次 API 请求的系统提示和工具定义可能达到 50-100K token，Claude Code 通过缓存和按需加载来控制这个开销。

子代理隔离与按需加载 Skill。 子代理在独立上下文窗口中处理研究，大文件读取不会进入主窗口，只有摘要返回。Skill 通过 CLAUDE.md 和按需加载机制管理，主上下文保持干净。

缺点

闭源、不可自部署。 Claude Code 是 Anthropic 的专有产品，Harness 是闭源的混淆 JavaScript。外部开发者无法轻易重塑系统。

系统提示冗长，上下文预加载重。 在你输入任何内容之前，CLAUDE.md、自动内存、MCP 工具名称和技能描述都已经加载到上下文中，大约消耗 7,850 token。对于垂类场景来说，这其中有大量不需要的通用约束。

大量代码由 AI 生成，维护难度高。 泄露源码约 51.2 万行 TypeScript，约 40 个内置工具。社区分析估计只有很小一部分代码构成 AI 决策逻辑，其余是运营基础设施。

安装配置门槛。 面向开发者设计，对非专业人员不友好。

OpenClaw 的优缺点

优点

开源、可自部署、模型无关。 MIT 许可，社区主导，可以完全自托管。OpenClaw Enterprise 明确设计为 vendor neutral，harness、模型和沙箱都可以替换为第三方方案或内部实现。

多渠道 Gateway 集成。单个 Gateway 进程同时支持 WhatsApp、Telegram、Discord、iMessage 等渠道，还有插件系统可以扩展 Mattermost 等。采用 Headless 架构，Agent 作为后台服务持续运行，适合消息驱动的自动化场景。

模块化架构清晰。 核心由 Gateway（网关）、Agent（智能体）、Skills（技能）和 Memory（记忆）四部分组成。支持多 Agent 并行协作，一个网关可对接多个 Agent，为“一人公司”模式提供技术基础。

Skill 生态规模。 Skill 数量庞大，每个 Skill 是标准化插件，通过 manifest.json 定义输入输出和触发条件。

缺点

安全风险突出。 这是 OpenClaw 最大的软肋。国家计算机病毒应急处理中心发现，多个仿冒 Skill 包包含恶意代码，安装后会在后台静默下载木马程序，窃取隐私信息甚至造成财产损失。

Token 消耗巨大。 有用户反馈单天 Token 费用高达 700 多元。

部署与运维门槛高。 部署需要注册 GitHub、英文阅读能力、懂 Git、会写 Python 配置环境。

执行反馈缺失，可靠性不足。 难以在执行过程中实时感知操作错误，实际使用中频繁需要人工干预，与“全自动化”的产品预期形成落差。

长期记忆与会话连续性。 2.0 版本才重点改进这一块，之前有明显短板。2.0 强化了历史会话搜索、对话分支、持续进度卡片，并引入了云会话、共享会话、团队角色等能力。

LangChain 也可以做状态图，为什么还要自研 Agent Loop？

【面试官】那 LangChain 也可以做状态图，为什么还要自研 Agent Loop？

【候选人】LangChain 确实能做状态图，LangGraph 也能做循环、条件分支、持久化这些编排能力。但“能编排”和“能把 Agent 的错误恢复闭环做深”是两回事。我们自研 Agent Loop，不是因为 LangChain 不能做，而是因为在这个垂类场景里，我们需要更细粒度的控制、更轻的依赖、更直接的结构化反馈闭环。

我分几层来说。

一、LangChain 偏重，依赖多，抽象层厚

LangChain 是一个通用框架，为了覆盖各种场景，引入了很多抽象和依赖。但在垂类 Agent 里，我们不需要那么多通用能力，反而会被框架的抽象层拖慢。比如错误处理、修复重试，框架本身提供的是通用回调、异常捕获，但我们要做的是：把成功、失败结果结构化封装，直接返回给大模型，让模型自己思考调整参数或者做弥补。

如果走 LangChain，往往要在中间加很多自定义组件、回调、状态字段，才能把错误信息完整、结构化地传回模型。
这反而让链路变长、调试变难，不如自己写一个 Loop 直接控制。

二、Agent Loop 的核心不是编排，是错误恢复闭环

LangGraph 解决的是“节点怎么连、状态怎么流转”。
但 Agent Loop 真正关键的是：工具调用出错之后，怎么把错误分类、结构化、反馈给模型，让模型决定下一步是重试、换参数、换工具，还是搜索相似文件。
这个闭环需要：

工具执行前：解析、校验、权限管理；

工具执行后：结果结构化，成功和失败都包装成模型能理解的格式；

模型拿到结构化结果后，自己分析、修复、调整。

自研 Loop 可以把这套逻辑直接嵌进去，不需要绕框架的抽象。
LangChain 要做同样的事，通常得自定义 Tool、自定义 Callback、自定义 Exception Handler，最后发现核心逻辑还是自己写的，框架只提供了个壳。

三、控制力与可观测性

自研 Agent Loop，我们可以精确控制：

每一轮注入哪些上下文、哪些经验；

工具调用的超时、重试、熔断、幂等；

错误怎么分类、怎么降权、怎么回滚；

日志、trace、评估指标怎么打。

LangChain 的抽象层会遮挡一部分细节，出问题时排查链路更长。
垂类场景对可靠性要求高，我们需要能追到每一次工具调用、每一次模型决策，自研更直接。

三、性能与依赖

LangChain 引入的依赖多，启动、打包、部署都更重。
自研 Loop 可以只保留业务需要的部分，轻量、启动快，也更容易做首 token 优化。
比如我们前面聊的首 token 优化，自研 Loop 可以精确控制哪些步骤并行、哪些延后、哪些走缓存，框架里反而不好改。

五、与业务深度结合
垂类 Agent 需要和业务规则、校验报告、经验库深度绑定。
比如 AutoSAR 场景，错误报告自带错误 ID、模块、描述，我们要用脚本解析、分类、粗筛经验，再结构化返回给模型。
这套逻辑和业务强相关，自研 Loop 可以直接把业务逻辑写进循环里。
用 LangChain 的话，要么把业务逻辑塞进 Tool，要么写自定义节点，最后框架反而成了束缚。

六、不是否定 LangChain，而是场景选择
如果是快速原型、通用场景、验证想法，LangChain / LangGraph 很合适，能省很多事。
但如果是垂类、生产级、对错误恢复和上下文管理要求高的 Agent，自研 Loop 更合适。
甚至可以是混合：用 LangGraph 做外层编排，核心 Agent Loop 自己写，把控制权留在自己手里。

记忆经验管理

记忆经验系统完整设计

【面试官】：记忆经验这块具体怎么做？讲下经验记忆系统完整设计。

【候选人】通用记忆和垂类经验记忆不一样。通用记忆，比如用户画像、用户偏好，用自然语言描述就可以了。我们这个记忆，或者说经验，主要是用来解决错误的。因此设计如下：

一、经验采集与结构化

最开始先和业务专家沟通，收集他们日常处理错误的修复经验，专家使用自然语言描述修复方案。

拿到自然语言描述之后做结构化预处理，增加错误 ID、模块、错误描述、解决方案、置信度这些字段。

做结构化的好处：校验报告原生就自带错误 ID、module 模块这些信息，解析错误报告之后，可以依靠 ID、模块先做一轮粗匹配，筛选部分经验，提升检索效率。

二、存储与索引式加载

存储层面借鉴 Claude Code 索引式加载机制，顶层总索引文档记录各个模块对应的经验文档位置；不同模块经验单独存一份文档。

解决错误的时候，匹配到对应模块，只加载该模块对应的经验，减少上下文占用。

而且拿到错误报告先用脚本做归类汇总；依靠错误 ID、模块先做初筛，减少加载进上下文的 token 数量。一般来说报告会有一两百个错误，归类汇总之后，再根据错误去检索相关的经验，而不是把所有经验全都一股脑地全部读取进来。

三、使用、新经验沉淀与冲突检测

使用过程中，内置经验不可能覆盖全部错误。遇到新的未见过的错误，专家可以继续提供新修复经验。

新经验同样先做结构化处理，之后再和已有的经验做比对，主要是冲突检测。冲突检测现在依靠提示词约束：除非大模型 100% 确定冲突经验里面哪一条有效、哪一条需要淘汰，否则不能自行决断，要把重复或者冲突经验抛出来，交给人工专家 check、决策。

四、置信度更新

修复会有多轮，第二轮、第三轮都会输出校验报告。

对比两轮校验报告，看应用这条经验之后哪些错误被解决、哪些错误新增。如果经验调用之后，对应错误被修复，就提高这条经验置信度，后续检索召回的时候排序权重会更高。

这样做经验沉淀，达到越用越聪明的效果，同时也能进行冲突检测的处理。

经验存储形式、经验更新、记忆过时、长短时记忆

【面试官】
经验具体是什么存储形式？经验更新，会不会出现记忆过时、短期长期记忆这类问题？经验数量膨胀、重复怎么处理？如何保证经验提取的质量？模型自己总结出来的经验，有可能是无效经验，这块怎么规避？记忆里边会涉及到遗忘之类的一些处理吧？

【候选人】
我按几个层面来说：存储形式、更新与质量、过时与遗忘、长短时记忆、膨胀与重复、模型自总结经验的规避。

一、经验的存储形式：结构化字段 + 分模块 Markdown + 总索引

经验是以结构化字段、分模块 Markdown 文件保存的，顶层有总索引文档。结构化处理时增加这些字段：**错误 ID、module、错误信息、修复方案、置信度**。

存储按业务模块分文件：不同模块经验分开存一份。好处是检索时可以依靠错误 ID、module 先做一轮粗筛——只检索加载对应模块的经验文件，减少上下文占用。

二、经验更新与质量保障

更新流程：沉淀新经验时，同样先做结构化处理，然后和存量经验做比对，做**重复、冲突检测**。目前依靠提示词约束：**除非大模型百分百确定哪条正确，否则不能自行决断，要把冲突、存疑的经验抛出来，交给人工专家 check 确认**。

这里有个隐含问题——模型自己总结出来的经验可能是无效的，怎么规避？现阶段我们主要还是依靠人工专家来校验判断。后续的规划是：先把模型总结的经验放到**候选暂存区**，先不正式落盘；后续多次调用，如果多次都能成功修复错误，再迁移到正式落盘区，成为正式有效经验。

三、置信度闭环——更新机制的核心

置信度更新是这样的：修复会有五轮甚至更多轮次，每一轮修复都会生成校验报告，也会记录这一轮检索、用到了哪些经验。两轮之间做对比：哪些错误被修复了、哪些是新增的。如果这条经验被调用、并且对应错误真的被修复，就**提高它的置信度**，检索优先级随之变高——我们每类筛选返回 **Top3 置信度最高**的经验，如果错误和几条经验都相关，置信度高的优先返回。这样就走通了“越用越聪明”的闭环。

四、记忆过时与遗忘机制

关于遗忘，其实我们设计的就是置信度机制——**可以理解为遗忘，但和遗忘还有一点点区别**。它有三个维度：**时间的维度、触发的频次、有效度**。

- **有效度**就是刚才说的：对比两轮校验报告，判断这条经验到底有没有用；有用且触发频次高，置信度就提高；
- **时间维度**上，一条经验比较久没被触发——比如一个月之前——就会**冷处理**，归档到文件夹或者数据库里。

不过要补充一点：我们和 To C 场景有区别。因为更偏向垂类项目场景研发使用，**不是按人的频率，而是按项目的频率**——项目一般是变化一个功能、有新

需求之后，会提供一份输入文件。所以衰减和过时的节奏是跟着项目走的，不是跟着人走的。目前我们主要就是用置信度的方式来管理。

五、经验膨胀与重复治理

目前垂类项目的经验总量还可控，还没有走到大规模膨胀那一步。但放到通用 Agent 场景，经验数量膨胀会带来上下文占用上涨，所以我们规划了四层方案：

1. **源头控制**：处理特定任务时，尽量只注入强相关经验，从源头控制注入数量；
2. **两层筛选**：如果经验库变大，第一层先做关键词粗筛，筛出强相关候选经验；候选集缩小之后，再交给大模型做语义相关性比对；
3. **合并去重**：按错误 ID、module 这些字段做经验合并逻辑；配合置信度——解决同类问题效果更好的经验，置信度更高、检索优先级更高；
4. **置信度+时间维度管理**：一条经验长时间没被触发调用，置信度降低、检索优先级下降，甚至归档到数据库做冷存储——低价值经验冷归档，这个目前还没有做，是规划中的。

我们目前**没有专门按照长短期记忆做分类，是按业务模块做层级划分**的。但如果对应过来看，其实有一个隐性的两层结构：前期和业务专家沟通得到的**预制内置经验**，Agent 启动就加载，相当于长期记忆的基线；运行过程**持续沉淀的新经验**，不断补充进长期记忆；而单次修复任务里检索注入的 Top3 经验，作用上更接近短期记忆。所以长短时的问题，我们目前是用“业务分层 + 置信度 + 冷归档”这套机制来覆盖的，未来规模上来了，可以再做显式的长短时分层。

记忆经验检索与召回

【面试官】：**记忆召回的时候，如何判断召回出来的记忆对当前任务是有效的、正相关的？**

【候选人】：核心设计逻辑一句话：**一切以工具的校验结果作为评价依据**。经验被调用之后，对比修复前后两轮校验报告——看新增了哪些错误、解决了哪些错误——如果这条经验真正解决了错误，就提升置信度；多次使用验证有效，检索时就优先召回置信度高的经验。加上时间维度：越近期的记忆权重可以更高，不要把所有记忆完全同等对待。

推广到通用场景——后置评估 + 多维打分。

任务执行完毕之后做**后置评估**：判断这条记忆/经验是不是帮助任务有效推进，有效就提升置信度。记忆还要叠加其他维度——类别、用户画像、时间、置信度，综合计算得分，衡量记忆和当前问题的匹配程度。

【面试官】：比如说怎么去判断匹配的经验，然后去判断这个问题应该用哪个经验？

错误报告里每个错误本身就有**错误 ID、模块、类型（容器/引用/参数）、自然语言描述**，经验和这个结构对齐——也涵盖所属模块、错误类型、通用化错误描述、解决方案。匹配流程三步：

1. **脚本归类**：写了一个解析错误报告的脚本——错误一般有几十上百个，先归类：同一个 ID 归一类，同一容器下的归一类。分完类一般留下**五到六大类**；
2. **分层过滤**：先做一轮过滤——错误 ID 完全不同的直接筛掉；错误 ID 相同的再判断；同一容器的、自然语言描述比较接近的也筛进来；
3. **Top3 返回**：一类错误筛选出 **Top3 条经验**。所以第一轮有 64 个错误的话，是**分五到六大类、按类统一去经验库检索，每类返回 Top3**——检索次数是按类走的，不是按错误个数。

【面试官】：经验里可能有不必要的操作、小偏差，怎么解决？通用能力匹配导致垂类场景支持不足、评估准确性怎么处理？

这部分修复一般不会一轮完成，我们设置**最大 5 轮**。以第二轮加载经验修复为例：第二轮修复生成第二轮校验报告，第三轮修复生成第三轮报告，**脚本对两轮报告做对比分析**——新增了哪些、解决了哪些、这些经验到底有没有用——据此对经验做一轮评估，调整置信度。

面试官担心的“通用能力对垂类场景支持不足导致评估不准”——我的回答是：这个风险真实存在，但我们把场景限得比较死，**已经给了模型相对充足且精确的上下文**：前后两轮错误明细、哪些修复了哪些没修复、每轮应用了哪些经验。知识不足的情况确实会遇到，下一步最好是垂类模型，但成本太高，所以**现阶段用“足量精确上下文”来缓解**。

【面试官】：面向 C 端个人助理，用户意图多变，话题会跳转，怎么解决记忆匹配错位？

意图多变、话题跳转的场景，我分三步走：

1. **记忆结构化打标签**：分类打标，区分科技类、生活类、工作类；
2. **两级过滤**：用户输入 Query，先用**轻量小模型做意图分类**——成本低、速度快——先做粗分类过滤，只取出对应类别的候选记忆；候选集缩小之后，再加一轮判断，交给模型评估候选记忆和当前 query 的相关性。如果对时间特别敏感，可以专门微调小模型做记忆匹配；
3. **多维打分**：类别、用户画像、时间、置信度，综合计算得分。

其它处理细节

评估的执行细节：校验报告是树状结构，**不完整读取整份报告**——专门的解析脚本先分类，再和上一轮对比。

失败回滚：对比后有修复效果就进入下一轮；**没效果甚至错误变多，就撤销本轮操作，恢复到上一轮状态；连续两次无修复就停止，转人工介入**。

查重：基本没有全量字符串匹配，现在是大模型做去重——新经验沉淀时把新旧经验放在一起，让大模型判断有没有重复、冲突，重新生成整理后的经验。冲突决策目前靠人工介入——实际使用看，大模型还不足以判断冲突时哪个是对的，我们还不信任它做决策，所以提示词里明确要求：**判断不了的要说出来，人工决策，不擅自写入或修改**。

负面标记：有的经验在这个项目沉淀的，换项目可能不适用。所以如果某条经验被调用后发现没解决问题、甚至导致更多错误，就记录下来、调低置信度，下次检索优先级降低。

人工介入后经验照样沉淀：超轮次终止后人工介入给方案，Agent 继续修复；这类新方案也会自动沉淀——遇到没遇到过的错误、经验覆盖不了的，新方案沉淀下去，同时和已有方案做对比。

Agent 记忆的边界与错误扩散控制

【面试官】：我们现在用 Claude Code 搭了基础的 Agent，但发现它会有“发挥过头”的情况，还会把错误操作记到记忆里，下次遇到其他厂商的芯片也套用错误的操作，像有传染性一样，某一次出问题后就反复犯同样的错，还会扩散到多个任务。你们项目里是怎么控制记忆的边界、避免这种问题的？

【候选人】：这个问题本质是：长期记忆被污染 + 记忆作用域过大 + 写入缺少准入。我会用「分类、校验、隔离、衰减」来做，但每一层都要落到具体机制。

一、先做记忆分类和作用域隔离
不能什么内容都往长期记忆里写。至少分：

短期记忆：单轮/会话上下文，自动过期。

任务记忆：当前任务用，任务结束归档。

项目/芯片级记忆：只对应某一类芯片、某一类任务，比如昇腾的经验只在昇腾任务加载，摩尔线程的只在对应任务加载。

全局通用记忆：只放经过验证的通用规范、通用操作，必须人工审核。

负面经验库：记录失败、导致新错误的经验，只做参考，不直接复用。

这样从根源上避免错误跨芯片、跨项目传染。

二、写入前做准入、冲突检测和人工审核
新经验写入长期记忆之前，先进候选暂存区，不直接生效。
准入规则：

来源可信度：专家确认 > 校验通过 > 模型自总结。

冲突检测：和已有经验比对，除非大模型 100% 确定哪条正确，否则抛给人工专家决策。

模型自己总结的经验，多次调用成功修复后，再迁移到正式落盘区。

写入时记录来源、作用域、版本、置信度，方便回滚。

三、执行边界：权限分级、沙箱、防越权
“发挥过头”要靠工程约束，不靠模型自觉。

工具权限分级：危险操作需审批，只读/可写分离。

沙箱执行：限制文件系统、网络、系统命令。

工具内部提前做防冲突、防重复执行。

关键操作走人工确认，不让 Agent 直接改长期记忆。

四、验证与更新：以校验结果为准，不靠模型自述
判断经验有没有用，看工具校验报告，不看模型说“我修好了”。

对比修复前后两轮报告：错误被解决就提升置信度；没解决甚至新增错误就降低置信度。

导致新错误的经验标记为负面经验，后续检索降权，只参考不直接复用。

连续多次失败就停止，转人工介入。

五、衰减与防扩散

时间维度：越久远的经验权重越低，长期不触发就冷归档。

频次维度：高频有效经验提权，低频无效经验降权。

成功率维度：多次验证有效的经验优先级更高。

跨域阻断：项目/芯片级经验不自动加载到其他域，避免错误扩散。

六、审计与回滚
每条经验要有来源、版本、作用域、置信度、调用记录。
出问题能回滚到上一版，能追踪是哪条经验、哪次调用导致扩散。
这样错误经验不会“像传染一样”反复犯，也不会扩散到多个任务。

Agent 工具

Tool-Call 工具调用整套工程设计、权限管理

【面试官】
工具的话，在里面设计工具都会包括哪些部分？工具加载、权限、调用和结果处理怎么设计？Tool Call 工具调用整套工程设计、权限管理。自定义工具怎么封装，怎么对接大模型？

【候选人】
我按几层来说：工具定义、工具加载、工具调用链路、权限管理、执行与结果处理、自定义封装对接。

一、工具定义：三要素 + 扩展属性
工具其实最核心的三要素就是名称、描述和参数，这是基础。
在此基础上，完善 Harness 的一套，就可以加上权限、风险等级、是否可并行，还可以加上是读操作还是写操作、是幂等的还是非幂等的这些属性。
这些权限、属性，是在调用工具的时候辅助判断这个工具到底赋予什么样的权限、怎么执行的。当然最核心的内容还是它的执行逻辑。

最简单的实现方式，就是先定义一个函数，把功能逻辑写到里面，再在外层封装一层——就像刚刚说的，把函数名称对应成工具名称，加上描述、参数信息，还有权限、是否可并行这些属性，封装成工具定义的形式传给大模型。
大模型用对应的参数去接收这些信息，就可以知道有哪些工具可以调用。

二、工具加载：按需加载，避免全量占上下文
工具加载不能一股脑全量塞进去。
垂类场景只需要特定工具，全量加载既冗余又影响效果。
可以借鉴 Claude Code 的懒加载机制。

三、工具调用链路：流式识别 + 格式校验
工具调用这套实现借鉴 Claude Code 整套体系。
普通做法，让大模型完整输出思考文本，再从输出文本里面解析工具调用指令。
流式输出的时候识别到工具调用指令就优先执行工具，节省等待完整输出的时间。

工具调用本质是 LM 加工具。但是想要工程化稳定持久运行，不能只做简单调用。
首先要做工具格式校验：大模型输出的 tool call，要校验格式是否合法，如果输出格式不对，是没有办法调用工具的。

四、权限管理：分级、人工确认
工具会和外部交互，破坏性操作类工具，需要人工确认；普通读取类工具权限可以放开直接运行。
权限管理是工具管理很重要的部分。

风险等级、读写属性、是否幂等，这些属性也用来辅助判断工具该赋予什么权限、怎么执行。

五、工具执行与结果处理：默认出错，结构化返回
最重要一点：要默认工具调用会出错。
无论成功失败，都要把结果做结构化封装返回给大模型。
模型拿到结构化返回结果，再去判断调用是否成功，区分是参数错误、网络连接错误、限流、权限问题，再决定下一步动作，修改参数、间隔重试等。
需要完整的错误处理闭环，不能简单输出指令直接执行，那只能跑 Demo。真正工程落地需要完整模块保证稳定运行。

六、自定义工具封装与对接大模型
工具最核心要素：工具名称、工具描述、入参参数。
在此之上还可以扩展字段，比如是否支持并行执行、读写属性、权限、风险等级。

不管是 OpenAI 还是 Claude，都是把整套工具定义结构体，通过 tools 参数传入大模型 API。
大模型就会识别可用工具，输出 tool call 调用指令。
我们把垂类自定义工具按照这套标准格式封装，通过 tools 参数传给模型，模型就可以触发调用。

工具返回结果过大怎么处理

【面试官】在 Agent 执行的过程里边，工具结果如果过大怎么处理？会频繁压缩等等一些事吗？

【候选人】通用 Agent 里面 read 读取文件，如果直接读取文件，那它返回的结果是非常大的。Claude Code 它会给工具设置一个 max 返回结果最大 token 数。如果在 token 范围以内那就直接全量读取，如果超过 token，那么可以先对读取的内容进行摘要提取，原始内容存储到文件。把摘要信息放到上下文里面传递过去。如果后面确定需要原始完整内容，再去读取原始文件。

我们这里的好处就是，我们知道这个垂类场景里面要读的内容是什么。
我们自定义读写这些核心工具，读的时候就根据错误信息，只去读最相关的 文件里面上下几十或者几百行区间的内容，而不是直接让它全量读取。
另外一点就是整个过程中生成的一些中间产物，如果后面需要再读，也不会全量直接去读取。比如说校验报告，我们会先用脚本先把报告进行汇总、归类，这样大模型读取的是预处理之后的文档，而不是原始完整报告。

工具调用返回结果和记忆系统的结合

【面试官】：工具调用返回结果，怎么和记忆关联？工具执行模块，要不要作为独立记忆单元，独立存储管理？

【候选人】：这个问题我会拆成三层来看：工具结果怎么进入记忆、工具执行模块要不要独立成记忆单元、怎么防止记忆污染和上下文膨胀。核心思路是：工具执行模块本身不直接等同于记忆，但它产生的原始结果、执行轨迹、提炼经验要分层管理，按需关联。

一、工具返回结果和记忆的关联方式

有两点思路。

第一，上下文窗口膨胀的大头，往往是工具读取的文件内容、工具返回结果。工具返回大内容，不要全部塞进上下文。可以把原始内容落盘保存到磁盘，上下文只放摘要加上文件路径，需要的时候再通过路径读取原始内容，这是和短期上下文管理结合。

更具体地说，工具返回结果可以拆成三层：

> 原始结果层：完整文件、完整报告、完整 API 返回，落盘或对象存储，不进上下文。
>
> 摘要索引层：摘要 + 文件路径 + 元数据，进短期上下文或索引。元数据包括工具名、调用 ID、任务 ID、项目/芯片、参数、状态、错误类型、时间、置信度。
>
> 经验层：如果这次工具调用成功修复了问题，经过校验确认后，才沉淀为长期经验；失败则进负面经验或候选区。

这样，工具结果不是直接写长期记忆，而是先落盘、再摘要、再评估、最后才决定要不要沉淀。

第二，全模块 Trace 追踪。
把每一次工具调用、入参、返回结果、执行轨迹完整记录。
一方面，出问题可以排查定位；另一方面，整套轨迹数据可以沉淀下来，未来用作强化学习样本。
把“怎么做任务”的完整过程记忆下来，遇到相似任务就可以检索过往执行轨迹，这一部分可以归属于记忆，更偏程序性记忆或情景记忆。

所以工具结果和记忆的关联，不是简单把结果塞进记忆库，而是：
原始结果落盘，摘要和路径进上下文，执行轨迹进 Trace，成功经验进经验库，失败经验进负面库。

二、工具执行模块要不要作为独立记忆单元

我的看法是：工具执行模块本身是执行器，不建议直接当成一个独立记忆单元。但它产生的三类东西，应该分开独立存储和管理：

> 原始结果：独立存储，按任务、工具、时间索引，只给路径和摘要。

执行轨迹：独立 Trace 存储，按调用链、任务 ID、会话 ID 索引，用于排查、回放、评估、RL 样本。

提炼经验：从轨迹中提炼，经冲突检测和人工审核后，进经验库，按模块、错误 ID、芯片/项目隔离。

如果做成独立子 Agent、独立记忆存储空间，会带来新问题：
子 Agent 内部执行轨迹需要一套共享空间，主 Agent 才能够访问读取子 Agent 的历史模块。
同时权限管控也要下沉到共享存储的文件目录层面。
所以更合理的是：共享存储 + 命名空间隔离 + 受控接口。主 Agent 通过接口读取子 Agent 的轨迹和结果，而不是全局一把梭。

三、和上下文管理结合，防止膨胀

如果任务模块很长，工具调用很多，全部塞短期记忆会撑爆窗口。
可以做策略：工具返回设置 max 阈值，内容小于阈值直接进上下文；超过阈值，只放摘要和文件路径，原始内容落盘。
后续信息不足，再根据路径调取原始文件，通过多次检索确认信息。

这套策略本质就是上下文窗口管理。
另外，脚本、结构化处理，会比交给大模型做压缩摘要的效果、效率更好。
比如校验报告先用脚本汇总、归类，再给大模型读预处理后的文档，而不是原始完整报告。

四、和长期记忆、经验系统的闭环

工具调用结果要和经验系统关联，关键是以校验结果为准：

工具调用后，对比修复前后两轮校验报告。

如果某条经验被调用后，对应错误被修复，就提升置信度。

如果没解决甚至新增错误，就降低置信度，甚至标记为负面经验。

新经验先进候选暂存区，多次验证有效后再正式落盘。

冲突时提示词约束：除非大模型 100% 确定，否则抛给人工专家决策。

这样工具结果、执行轨迹、经验记忆就形成了闭环：
调用 → 结构化返回 → 落盘/摘要 → 轨迹记录 → 校验评估 → 经验沉淀 → 下次检索复用。

多工具并发调用的冲突处理

【面试官】：那你有没有遇到过这种情况：多个工具并发调用，比如说多个工具同时修改你的配置选项，这种冲突的解决方式你有做兼容吗？

【候选人】：这个其实也是借鉴了 Claude Code 的思想。首先我们会对工具做分类：像这种“无害的”、只读的，不会修改文件、不会新建文件，只是读取，它的风险等级就比较低，这种我们就可以让它并行去读很多个文件。 但是像新增、编辑这类操作，风险等级就相对较高。这种情况下，借鉴 Claude Code 的思路，我们项目里也不会让它同时改写多个文件。如果强制要并行写多个文件，那就需要做锁的保护，不然很容易发生冲突——这个改完那个又改，版本不可追溯，修改内容也可能互相冲突。 我们现在其实是，写操作的时候没有让它并行去写。

此外要有冲突检测与恢复机制，写之前：检查版本、检查依赖、检查权限。写之后：校验结果，对比前后状态。

工具调用的错误分类与容错处理

【面试官】：还有就是你说执行底座的时候，提到了一些错误优化对吧？比如说用工具的时候遇到卡死、超时，有没有区分这种是可重试的错误还是不可重试的错误？另外有没有遇到过一些案例，是大模型解决不了、不干预不行的？

【候选人】：这个算是一些边界情况，实际项目里相对少见，但设计执行底座时必须考虑。我会先把错误分类，再决定是自动重试、让模型自修复，还是直接转人工。

一、先按来源和可恢复性分类
错误不能混在一起处理。我会分：参数错误、权限错误、网络错误、限流、超时、卡死、锁冲突、版本冲突、工具内部 bug、模型决策错误。
再按可恢复性分：可重试、不可重试、必须人工干预。

二、可重试错误：退避、幂等、熔断
比如网络抖动、限流、临时锁、资源暂不可用。
策略就是指数退避重试，设最大次数；工具尽量幂等，防重复执行；重试失败后结构化返回给模型，让它决定换参数、换工具还是等待；连续失败就熔断，避免一直打同一个坏工具。

三、不可重试错误：直接返回，让模型换路径
比如参数格式错、权限不足、路径不存在、版本冲突、工具内部 bug。
这类不重试，直接结构化返回错误类型，让模型重新解析参数、换工具、泛化搜索相似路径，或者终止转人工。
关键是错误信息要足够具体，不能只返回“失败”。

四、卡死和超时：专门处理
卡死和超时不能只靠模型判断。
工具调用设超时，超时直接中断；心跳机制监控长任务；卡死后 kill 进程，回

滚到上一状态；标记该工具不健康，后续降权或熔断；如果是写操作，要保证可回滚、可重放。

Skill

三段式加载的流程

【候选人】所谓的三段式加载。 三段分别是啥呢？第一段是 metadata，描述这个技能的名称、作用、关键触发点是什么，一般在对话的时候会加载到系统里，让 Agent 根据用户的描述去匹配是不是触发对应的技能。 第二段是匹配到了之后，再加载正文内容——技能的正文内容，用标准的执行步骤描述技能要做的操作步骤。 第三段是，如果正文内容要求精简，不要超过 500 行，但有的技能很复杂，或者需要参考文档，就进入第三段 Reference，加载参考文档的信息进来。这是三段式渐进式加载的核心流程。也可以加一些脚本，直接去执行具体的操作。

其实整个过程就是思考-执行-观测的反复循环：先让大模型拿到信息，先思考，觉得要执行动作了就调用工具；工具执行完之后把结果返回给模型，去观测结果是什么样、有没有错误；观测之后觉得还要调用其他工具、做其他工作，就继续循环，直到觉得执行动作差不多结束了，可以给用户最终反馈了，就结束循环，给用户输出最终的汇总结果。

不知语冰

上下文管理

如何理解 Agent 的上下文管理

我理解的上下文管理，本质上是在有限 context window 里，为模型构造当前决策所需的最小工作集。它不是把所有项目文件、历史记录和经验库都塞给模型，而是根据当前任务阶段选择规则、用户目标、业务证据、历史状态、工具 schema 和输出约束。好的上下文管理要做到四点：信息足够、证据可追溯、噪声可控、状态可回放。尤其在工程 Agent 中，上下文管理直接影响模型是否会幻觉、是否重复犯错、是否错误套用历史经验，以及是否能安全执行写操作。

上下文里通常包含哪些内容

通用 Agent 上下文一般分为几类：第一是系统和项目规则，比如 system prompt、AGENTS.md、SKILL.md；第二是任务输入，比如用户需求、配置、附件和路径选择；第三是业务知识，比如领域规则、模块知识和历史经验；第四是当前证据，比如 report 摘要、文件片段、工具结果和校验结果；第五是运行状态，比如上一轮做了什么、是否回滚、错误是否减少；第六是工具上下文，也就

是当前允许模型调用的工具 schema。核心原则是：当前必须决策的信息进入 prompt，大文件和长历史放 artifact，通过工具按需读取。

上下文窗口管理与压缩机制

【面试官】：对话越来越长的话，上下文不会爆吗？你们有没有做压缩机制？

【候选人】：这个项目里真正的用户对话消息其实很少，占用空间不大，真正占满上下文窗口的，是背景知识和工具执行返回的中间产物。
所以我一般会先理清上下文都包含哪些组成，再针对性优化。

一、先定位膨胀大头
这个项目是自动执行的 ReAct 循环，不是持续累积的多轮对话。
只有修复失败、达到最大轮次后，才会人工输入新的经验指令。
所以上下文膨胀的压力主要来自文件读取和工具返回的中间产物，而不是对话累积。

二、分层优化

System Prompt 精简
通用 Agent 系统提示词大而全，垂类只保留角色、业务流程、核心背景、必要约束，减少 token 占用。

工具层优化
通用工具定义占上下文 10% 到 15% token，精简掉业务用不到的工具集合，只保留垂类需要的特定工具。

重点处理多轮工具调用带来的膨胀
这块是上下文膨胀大头。
① 校验报告，不用直接喂原始完整报告。先用脚本做分析、聚类、预处理，过滤冗余内容，只保留核心错误信息，减少传入内容体量。
② 自定义读取文件 的工具，因为文件是树状结构，不读取整个完整大文件，只读取相关的局部片段。
③ 对读取文件的工具做返回限制，模仿 Claude 的设计，设置最大返回行数，超量就截断或转存到文件，不全量读取。

记忆经验侧优化
经验分模块存，有顶层索引文档。
遇到某模块错误，只加载对应的经验文档，不会把全部经验一次性载入上下文。
这样也能减少上下文窗口压力。

二、压缩机制

如果 System Prompt、工具定义、消息等加起来超出窗口，就启动对话压缩机制，对历史内容做精简。

上下文压缩不是简单截断，而是结构化降维。比如 report 不能全量塞进去，应先按错误码或语义 pattern 聚类；大量错误实例只保留代表样例；经验库只展开命中的经验；历史 trace 只保留上一轮关键失败和更早轮次摘要；脚本日志只保留 stdout/stderr excerpt；工具长结果写 artifact，只回传摘要和路径。压缩时必须保留对象路径、行号、operation_id、diff 和 rollback 原因，因为这些是后续决策和审计的证据链。

但因为这个项目是自动执行的 ReAct 循环，对话消息本身很少，所以压缩机制更多是兜底，主要压力还是靠工具返回限制、脚本预处理、局部读取、分模块加载来解决。

上下文组装的工程化问题

【面试官】
那回到上下文组装的工程化问题。C 端用户问题千奇百怪，我们怎么预先定义几套 Prompt，怎么根据用户意图去匹配？你怎么看这件事？
你说按风险敏感分两套 Prompt，但我有个疑问：C 端 Agent 的前置内容审核，是不是直接就能解决敏感问题？直接把敏感内容屏蔽掉不行吗？

【候选人】
我分几层来说：Prompt 组装、前置审核的边界、ReAct 首 token 可控的思路。

一、Prompt 不要预定义几套，而是组件化动态组装
我不建议预先定义几套完整 Prompt，因为 C 端问题千奇百怪，硬编码几套很快就不够用。更好的做法是：定义 Prompt 组件库，按意图动态组装。

组件可以包括：

系统角色：通用身份、语气、能力边界。

安全基线：所有场景都加载，防止泄露、越狱、敏感内容。

场景约束：技术类、生活类、问答类，各自只加载相关约束。

工具说明：只加载当前任务需要的工具 schema。

输出格式：按场景定义，比如 JSON、Markdown、纯文本。

用户画像：从长期记忆里取，动态注入。

然后加一个轻量意图分类器，输出几个标签：风险等级、场景标签、是否需要工具、是否需要 ReAct、上下文预算。
简单问答直接走低延迟模型；复杂任务才走 ReAct。

这样不是靠几套 Prompt 省 token，而是靠动态组装和路由，减少不必要的上下文，同时保证安全和场景适配。

二、前置审核不能替代 Prompt 分级
风险等级不是非黑即白的，不是只有“高危”和“完全安全”两级，它是有梯度的。

前置内容审核解决的是“用户输入里有没有明显敏感内容”，但很多风险是在模型输出侧、工具调用侧、上下文组合后才出现的。
比如用户问的是正常问题，但模型可能因为上下文里带了敏感信息而泄露；或者用户诱导模型绕过约束。
所以审核要分层：输入审核、输出审核、工具调用审核、上下文脱敏。
Prompt 分级是另一道防线，两者互补，不是替代关系。

多 Agent

我理解的多 Agent 架构不是简单地开多个模型并行，而是把一个复杂任务拆成“主 Agent 负责目标、状态和决策，子 Agent 负责局部问题求解”的协作体系。主 Agent 是 orchestrator，负责接收用户目标、拆解任务、选择子 Agent、传递上下文、合并结果、做最终决策和兜底；子 Agent 是 bounded worker，只拿到完成局部任务所需的最小上下文，输出结构化结论或候选方案。

典型生命周期包括创建、上下文传递、执行、结果回传、主 Agent 汇总、子 Agent 销毁。

这里最关键的是边界：子 Agent 不应该直接拥有全局写权限，也不应该独立决定最终状态，它更像是一个专业分析器或执行器，主 Agent 才是最终责任主体。这样做的收益是降低上下文污染、支持并行、减少单 Agent 认知负载，并能把审查、修复、验证、记忆沉淀等职责拆开。

多 Agent 缓解上下文膨胀的原理

【面试官】
那多子 Agent 为什么可以缓解上下文窗口压力？

【候选人】
上下文来源主要分几部分：System Prompt、Skill 工具定义，占比最大的是多轮工具调用返回的文件内容、工具执行结果。
多子 Agent 的核心原理，其实就是上下文隔离：每个子 Agent 有自己独立的上下文窗口，内部执行过程不进入主 Agent 的上下文。

一、先定位膨胀大头
真正占满上下文窗口的，不是用户对话，而是工具调用返回的文件内容、校验报告、中间产物、错误重试记录。

这些内容如果全部留在主 Agent 里，主 Agent 上下文会迅速膨胀，还会带来 Lost in the Middle、注意力稀释、错误扩散。

二、子 Agent 的核心原理：上下文隔离
子 Agent 的思路，就是把一块复杂任务打包交给子 Agent 去完整执行。
主 Agent 不需要看子 Agent 内部每一轮的工具调用、读取的大段文件内容，只接收子 Agent 汇总之后的最终结果。
这样子大量中间执行信息不会带入主 Agent 的上下文，就可以降低主 Agent 上下文占用。

更准确地说：
子 Agent 内部可以有自己的 ReAct 循环、工具调用、文件读取、错误重试、上下文压缩。
这些中间状态留在子 Agent 自己的上下文里，子 Agent 完成后只回传结构化摘要。
主 Agent 只维护任务级状态，不维护执行细节。

三、主 Agent 和子 Agent 传递什么
主 Agent 传给子 Agent 的，是任务契约：模块标识、目标、前置结构化信息、权限、输出格式、预算限制。
子 Agent 返回给主 Agent 的，是结构化结果：摘要、修复方案、错误数变化、置信度、文件路径、Trace 引用、失败原因。
原始大文件、完整校验报告、详细执行轨迹，落盘或放共享存储，主 Agent 需要时再按路径读取。
这样主 Agent 上下文里只有摘要和路径，没有大段原始内容。

四、项目里的落地
我们项目里面有六个模块，就会启动六个子 Agent 分别配置模块。
子 Agent 之间工具集合大体相同，只需要传入不同模块标识、前置提取出来的结构化信息。
子 Agent 的启动是借鉴 Claude 的工具调用方式，通过 Tool Call 来触发。
每个子 Agent 负责自己模块的读取、校验、修复、经验检索，主 Agent 只汇总各模块结果，决定下一步是继续修复、重试还是转人工。

子 Agent 的全流程

【面试官】
主从 Agent 的完整执行模块，从一个请求进来以后，当你决策开始要走这条路以后的完整逻辑讲一讲。执行完之后呢？失败了会怎么做？无限迭代还是有控制？控制完后如果还是失败这些怎么处理？

【候选人】
我按几个层面整合一下：整体设计、创建与启动、上下文传递、工具差异化、执行与结果回传、失败处理、上下文膨胀解决、调度、销毁与经验沉淀。

一、整体设计：借鉴 Claude 主从 Agent 通信模式，做简化
一个请求进来以后，这块我们主要是借鉴了 Claude 的主从 Agent 通信管理模式。
当然它的通信模式有很多种，我们做了简化。

比如说现在我们要配置 C 模块，这是当前的任务。
主 Agent 前面已经生成了一些中间产物文件，当接收到“配置 C 模块”这个任务的时候，主 Agent 会把任务，加上任务需要的中间产物文件路径、其他依赖文件的路径，都放到提示词里，传给子 Agent。
子 Agent 也有自己的 System Prompt，我们用 Markdown 格式写的，定义了它的核心职责，然后就进入子 Agent 的执行阶段。

二、子 Agent 的创建：建立受限执行边界
创建子 Agent 的关键不是“开一个新会话”，而是建立一个受限执行边界。
主 Agent 应先判断任务是否满足子 Agent 条件：是否可并行、是否有清晰输入输出、是否需要专门知识、是否能限制权限。
如果满足，就生成 task spec，例如 task_id、role、objective、inputs、allowed_tools、read_roots、write_roots、expected_output_schema、budget、deadline。

三、上下文传递：只传必要内容
比如说配置场景里，现在的任务是你要配置哪个模块，然后把任务、任务所需要的上下文信息、以及任务需要的文件路径，通过参数的方式传递给子 Agent，然后启动子 Agent 去完成配置。

上下文传递时只传必要内容，比如当前报错片段、匹配经验 ID、候选对象路径，而不是把整个工程和所有历史塞进去。
子 Agent 回复时返回 JSON 或表格化结论，主 Agent 负责合并和冲突处理。

四、工具差异化：按场景区分
不同的任务传不同的工具，我们现在主要是根据场景去区分的。
比如说配置 C 模块的时候，传的是统一的一套工具；去解析、查错的时候，用另一套工具。不同的场景对应不同的工具集。

五、执行与结果回传
举个例子，多子 Agent 是用来配置模块的，我们以单个模块为例，子 Agent 的任务是配置 C 模块，它会拿到需要的中间产物文件的位置。
然后有一个类似启动器的工具，借鉴了 Claude 的工具设计，把这些信息作为参数传进去，相当于启动对应的子 Agent，子 Agent 就自己去处理对应模块的配置就可以了。

完成之后，把完成的结果总结出来，返回到主 Agent 这边。
主 Agent 就可以知道当前完成的状态是什么样子，目前配置的文件在哪里。

六、失败处理：目前靠校验反馈，后续要补控制
我们现在其实甚至可以说，目前没有专门的失败处理机制。

主要是子 Agent 生成文件之后，后面会调用工具去做校验，评测，主要是靠校验结果来判断前面的步骤有没有成功。

但更完善的做法应该是：

子 Agent 内部有最大轮次、超时、预算限制，不能无限迭代。

失败后结构化返回错误类型，区分可重试、不可重试、需人工。

可重试的走退避重试；不可重试的让模型换路径。

连续失败就熔断，转人工介入。

失败的配置经验沉淀到负面经验库，调整置信度。

垂类 agent（结合具体业务，已脱敏）

垂类 Agent 对比通用 Agent，做了哪些改造，踩过什么坑

第一点，也是首要的，是**业务抽象与流程拆解**。现在 通用 Agent 的架构已经很成熟，但在垂类业务里能用、好用的必要条件是业务与 agent 架构的有效融合。 做 Agent 之前，先要把业务流程拆透：哪些步骤是固定的、确定性的，就用确定性的代码和脚本硬编码实现；哪些环节需要判断、推理，再交给大模型用 Agent loop 去做。

然后改造 System Prompt。通用 agent loop 需要面向全场景、全用户，System Prompt 很全面，包括安全校验、网络防护、各种边界约束。但垂类场景相对固定，所以系统提示词不需要特别复杂，只需包含核心的角色以及职责边界，执行任务的规则和约束、任务流程以及最相关的背景知识。精简 System Prompt 之后，既减少了 Token 消耗，也降低了上下文窗口的占用，减少幻觉。

接着便是工具。优化点有 2 个，一是工具的数量，二是自定义工具的管理。通用 agent loop 有几十个工具，工具 schema 大概会占 10% 甚至更多的上下文窗口，因此可以省去垂类场景里完全用不上的工具。此外，通用 agent 是在外层 harness 里编排 ToolUsePipeline 统一管理工具的解析，校验，权限，执行，结果返回的，但在垂类里可以自定义工具，并在工具内部对处理任务执行动作进行约束管理，实现工具的双重管理，使得工具执行的错误出现次数更少，效率和成本更优。

再者是记忆经验体系。通用 agent 的记忆更偏向用户画像，项目核心信息，历史关键操作等，用自然语言描述。但是垂类 agent 的记忆更偏向于专家行业经验，因此需要对经验进行结构化处理，添加置信度，并分模块存储，方便检索。此外，在使用过程中应该不断沉淀新的经验，并进行去重和冲突检测，达到越用越聪明的自进化目的。

最后便是上下文管理，除了上面所提的系统提示词优化，工具 schema 优化以及记忆经验体系，还有一个重要的优化点便是把工具的输出内容比如读取的文件等先进行预处理，将原始内容分类，大小限制，结构化汇总，再组装到上下文里，也缓解了上下文窗口的压力。

如何迁移 Agent 方案

Agent = 模型 + Harness，模型只负责想，Harness 负责做 aliyun.com+1。我们这套框架里真正可迁移的资产是 Harness 层——Agent Loop、工具调用、错误闭环、记忆系统、上下文管理——这些在制造业、有标准化流程的业务里都是通用的；**真正要适配的是业务层的抽象：把传统模块区分出来，哪些是确定性动作，哪些是有变量、需要大模型介入的动作**。想清楚这一点，迁移就变成"换插件"而不是"重做系统"。

不管是迁移到 GPU 迁移、家庭助手还是其它制造业场景，我都用同一套方法：

第一步：业务抽象和流程拆解。 先拿一个真实的人工操作案例，把人工的每一步拆解出来——**区分"完全固定、可以脚本化"的步骤，和"需要分析判断、需要大模型介入"的步骤**。固定步骤直接用自动化脚本实现，需要分析差异、判断适配方案的部分再交给大模型。这一步是迁移的灵魂——**很多时候根本没必要训练一个垂类模型，只要把一个领域里流程的标准动作固定下来，需要大模型介入的时候，把经验、背景知识以 System Prompt 或其他方式给到这个循环，它就可以在这套相对固定的 Harness 层面跑起来**。

第二步：路由与前置知识注入。 给 Agent 注入领域的前置背景知识——芯片架构规范也好、家庭生活常识也好——针对不同的子场景、不同的输入类型，路由到对应的处理模块。

第三步：工具融合。 把已有的适配脚本、编译工具、检测工具、现成 API 都封装成 Agent 的工具，让 Agent 调用工具执行具体操作，**而不是让大模型凭空生成代码**。

第四步（最核心）：反馈闭环。 任务完成后要有验证、评价机制，判断结果对不对。有反馈就能做自修复循环——**这是 Agent 能持续迭代的基础，也是我们前面聊的置信度机制、校验对比机制能跑起来的前提**。

可观测追溯体系

如果 Agent 出问题，怎么排查？

分四步说：**先定位是哪一层的问题，再靠 Trace 回放查，然后不只看结果还要看过程，最后让数据反哺迭代。**

第一步：先分层定位，不盲目翻日志。

Agent 出问题，一般先区分是哪一层的故障：

层	典型问题举例（结合 AutoSAR 项目）
模型层	意图识别判错模块、抽取信息遗漏、输出幻觉
编排层	状态流转错误、子 Agent 调度异常、并行冲突
工具层	工具调用失败、入参格式错、返回结果解析异常
数据层	上游 Excel 本身有误、结构化提取的字段错
工程层	API 超时、权限校验失败、并发问题

先分类，能避免拿到问题就漫无目的找。

第二步：全模块 Trace 回放。

完整的追溯评测体系，属于工程落地的必要部分。说实话，我们前期优先跑通业务流程，Trace 没有系统性完整实现，现在正在补充。目标形态是：**整套思考模块、每一次工具调用、工具入参、工具返回结果，全部做日志完整记录**，每一步用唯一 Trace ID 串联，形成完整执行模块。出了问题可以逐步回放，定位到具体是哪一步思考、哪一次调用出的错。定位到之后，还可以用 Trace 做**确定性复现**——固定输入、Mock 工具返回，把偶发问题稳定重现出来再修。

第三步：不只看结果，还要看过程。

不光要看最终任务成功失败。同样完成任务，有的调用 3 次工具，有的调用 7 次工具，过程效率差异很大。所以我们除了成功率，还关注**过程指标**：修复轮次分布（是否都用满了五轮上限）、工具调用次数、token 消耗、经验命中率。过程指标能提前暴露“结果对了但路径退化”的情况，比如经验没命中、每次都在靠模型硬推。

第四步：Trace 数据反哺迭代。

日志 Trace 数据，后续还可以用来提炼经验，或者作为强化学习的样本，让 Agent 不断迭代进化，从“业务小白”成长为领域专家。拿我们项目举例：修复失败的 case 会沉淀成新经验；修复成功、置信度高的经验，下次同类错误直接命中，修复轮次就降下来了。同时每次迭代后跑固定的**回归评测集**，保证新版本没有把原来能修对的问题改坏。

线上服务排查场景的 Agent 设计

【面试官】
我们来聊一个场景吧，现在线上服务出问题，设计一个可以自动查找问题的 Agent，你能具体说一下线上服务的问题有哪些常见的，还有结构能不能再稍微

具体一点？就是这种在线服务的场景。最终使用的话，可以达到什么样的效果？定位的指标嘛。

【候选人】
我先理解一下您说的情景：是不是相当于我现在有一个服务，服务会出现各种问题，需要用一个 Agent 判断这些问题出在哪里、有哪些错误？是这个意思吗？

因为这种服务的话，可能逻辑本来就很多、比较复杂，开发过程中会出现一些问题，导致服务或者工程上的问题。这个场景其实是非常细化、非常具体的一个场景，我觉得 Harness 底层其实可以复用，核心的设计优化点应该放在几个地方。

一、先看线上服务常见问题
线上服务的问题，一般可以分成几类：

服务不可用：进程挂了、端口不通、健康检查失败。

请求异常：报错、超时、限流、熔断、重试失败。

性能问题：响应变慢、CPU/内存/IO 瓶颈、线程池打满。

依赖问题：数据库、缓存、消息队列、第三方接口异常。

这些问题的共同点是：现象很多、日志量很大，不可能全量读代码、读日志，所以 Agent 一定要有定位逻辑。

二、整体结构：复用 Harness，重点做垂直能力
底层 Harness 可以复用，比如工具调用、权限、上下文管理、错误恢复、记忆管理。
核心优化点放在这几个地方：

1. 测试能力：给 Agent 写一个 Skill
要给 Agent 写一个 Skill，告诉它怎么去测试这个服务。
测试的时候输出测试报告，再对测试报告做分解分析，定位可能出问题的点。
要写一套规范告诉它怎么写测试，让它针对服务去做测试。
不是让它凭空猜，而是用测试结果、报错、指标、日志来做证据。

2. 记忆经验管理：这是非常重要的
就拿人来说，有经验的人排查问题，速度肯定比没有经验的人快。
所以要设计一个好的记忆经验系统：可以根据测试结果，或者实际现象里反映出的报错、用户反馈的问题，去匹配相关的经验——之前有没有遇到过、其他项目有没有遇到过类似的问题，问题定位点在哪，解决方案是什么。
经验要结构化存储，按模块、错误类型、现象、根因、解决方案、置信度来管理。
调用后如果真正修复了，就提升置信度；没修复甚至引入新问题，就降权或进负面经验库。

3. 定位逻辑：要有先后顺序
因为在线服务的日志量可能会比较大，不可能全量去读代码、读日志。
所以应该先对整个服务按模块、按功能做拆分。
好处就是，可以根据测试报告或者反馈，先定位到具体的模块，有的放矢，而不是全面扫描整个代码仓库——工作量非常大，也非常耗费资源。
定位到相对具体的模块之后，效率会高一点，消耗也会少一点。
然后再在模块内部，按日志、指标、链路、变更记录、配置差异逐步缩小范围。

三、完整执行流程
一个告警或问题进来以后，可以这样走：

收集现象：报错、日志、指标、链路、变更、用户反馈。

服务建模：按模块、功能、依赖关系拆解。

粗定位：根据现象匹配模块和经验。

生成假设：可能是什么问题。

调工具验证：查日志、看指标、跑测试、对比配置、检查发布记录。

定位根因：确认是哪一类问题。

给出修复建议或自动修复。

修复后验证：服务恢复、指标正常、错误消失。

沉淀经验：成功经验进经验库，失败经验进负面库。

四、最终指标
首先就是修复能不能成功，这是硬指标——部署之后出现了报错、出现了 bug，最终目的是把问题修复，这是修复成功率。
另外一个就是效率：人工修复一个 bug 需要多长时间、多少工时，现在需要消耗多长时间、多少资源，这是衡量指标。
还有过程指标，也会体现到成本控制：修复问题的时候，能不能按照最优或者相对更优的路径、工具去执行，拆解是不是按照更优的路径去执行。就是从这几个维度去衡量。

评测

如何评价 agent 的效果呢

把“agent 效果”拆成可评估的几层：最终有没有完成任务、过程是否高效、在边界条件下是否稳定。这样比只看回答文本更接近真正的 agent 能力。评价 agent 效果，核心不是“回答像不像人”，而是：

在给定任务、约束、工具和成本下，agent 是否稳定地把事办成了

所以要把评估拆成几层看，而不是只看最终文本好不好看。

1. 先看结果层
这是最重要的一层，直接问：

- 任务有没有完成
- 完成得对不对
- 结果有没有可验证的错误
- 有没有产生副作用

不同 agent 的结果指标不一样：

- Coding agent，看测试是否通过、补丁是否引入回归

这一层最该看的是 success rate，而不是单条回复质量。

2. 再看过程层
agent 的价值很大一部分体现在“怎么做事”，所以要看过程是否高效、可靠：

- 工具选得对不对
- 调用参数对不对
- 有没有不必要的多余步骤
- 能不能在失败后自我纠正
- 会不会反复试错、死循环

常见过程指标有：

- tool call precision / recall
- 平均轮数
- 平均工具调用次数
- 自我修复率
- 任务中断率

这层能看出 agent 是“聪明”还是“瞎忙”。

3. 看效率层
如果结果不错，但太慢、太贵，也不能算好 agent。

常看这些：

- TTFT，首 token 时间
- 总任务时长
- token 消耗
- 工具调用成本
- cost per successful task

对成本敏感的场景，最后这个指标特别重要。
不是“每次调用便宜”就行，而是“每次成功完成任务的综合成本”要低。

4. 看鲁棒性
好 agent 不是只会做标准题，而是在变化和噪声里还能跑。

要专门测：

- 改写后的同义问题
- 信息不完整的输入
- 工具超时或失败
- 长上下文
- 恶意提示词干扰
- 边界案例和异常输入

这里要特别关注长尾失败，因为 agent 往往不是平均值差，而是某些场景会突然崩。

不如语冰

开放性问题

那你觉得 Agent 未来发展方向是什么？

我思考有这么几个方向。 一个是 Agent OS。现在 Agent 大多是单个 Agent 完成一项任务，以后真正大规模落地到真实工作场景，会像人类团队协作一样。Agent OS 做调度，把多个不同角色的 Agent 组织成数字员工团队，Agent 之间相互沟通协作，这样生产力会得到极大提升，也能够规避一部分幻觉，提升指标。另外一个方向就是端侧 Agent。手机是现在人最高频使用的终端，不可能所有 Agent 任务全部依赖云端。在手机以及各类嵌入式设备上面部署效果良好的端侧 Agent，我觉得潜力非常大。

Agent 自进化思路，技术方向思考

【面试官】：那你觉得现在这套模式，做 Agent 自进化会面临哪些问题？你还了解哪些改进思路？
【候选人】：我现在思考有两条路径可以做自进化。 第一种就是现在做的经验沉淀模式：不断沉淀业务经验，让垂类 Agent 从业务小白，靠经验积累成长成领域专家。 第二种是全模块轨迹数据收集。把每一次任务完整执行轨迹全部记录

保存，基于轨迹数据做强化学习，优化 Agent 任务执行能力。 另外还有一种思路，Agent 和人工交互完成新任务之后，把处理流程沉淀成 Skill，内化成自身能力。 我也在思考，垂类 Harness 底座，如果搭配垂类微调小模型，效果有可能优于直接调用 GPT-6、GPT-5.1 这类大模型，同时还可以降低成本。

Agent 什么时候需要做模型微调

【面试官】：你认为什么条件下需要去训练微调一个垂类模型？

【候选人】：什么时候需要做垂类模型微调： 第一，通用大模型拿不到行业私有数据，比如制造业、医疗这类行业内部私有业务知识，通用模型没有见过，这种情况垂类微调就很有必要。 第二，业务数据涉及隐私、数据安全，希望本地部署，不调用外部 API，适合微调垂类小模型。 第三，如果只靠通用 Agent+Harness+领域知识库，已经达不到业务预期效果，就考虑微调。 第四，成本角度，如果业务需要高频调用高价大模型，综合成本算下来，微调部署小模型会更划算。 第五，响应速度要求高，调用外部 API 有网络延迟，本地微调部署模型可以提升响应速度。 综合这几个维度权衡。

多人协作怎么做？怎么保证迭代质量？

我们团队协作分两个阶段。 第一阶段：模块化分工，有人负责记忆经验模块，有人负责自定义工具，有人负责上下文管理，各自开发对应模块。

第二阶段，团队对 Harness 整套体系有统一理解之后，切换新协作模式： 现在写代码不是核心卡点。每个人先调研对应模块业界主流实现、本业务场景特点，输出调研汇总文档。 团队几个人一起开会讨论文档，共同梳理，确定完整架构、输入输出、细节逻辑。 架构和细节全部讨论清楚之后，再把完整设计交给 AI（Claude）生成代码。 生成代码之后，人工做功能 Review，验证是否符合设计预期，同时检查有没有可优化点。

我的体会：如果只是简单丢一个需求给 AI 写代码，AI 很容易产出和预期偏差很大的代码，幻觉多，后续 Review 成本极高。 **前期架构、逻辑、细节梳理的越完备，AI 产出代码质量越高，写代码本身反而很快。** 两到三个人小团队，做调研、集体讨论碰撞思路，集思广益，效率是比较高的。